Course Generation Algorithm - Full Report

Generated: 2026-07-11T09:03:56.809411+00:00

Course section: 4.1 Software Engineering Modules

Module: Software Construction, Architecture and Engineering (2025-MCSSE-SE-01)

Step 0.5 - Lecture plan

Source: generated | Lectures: 10

1. Introduction to Software Engineering and Object-Oriented Fundamentals (90 min)
2. Inheritance, Polymorphism and Dynamic Binding (90 min)
3. Genericity and Type Systems (90 min)
4. Design by Contract and Software Reliability (90 min)
5. Fundamental Design Patterns I: Creational and Structural Patterns (90 min)
6. Fundamental Design Patterns II: Behavioral Patterns (90 min)
7. Software Architecture and Large-Scale Program Structure (90 min)
8. Requirements Engineering Fundamentals (90 min)
9. Agile Methods and Modern Development Practices (90 min)
10. Software Engineering Process and Quality Assurance (90 min)

Lecture 1: Introduction to Software Engineering and Object-Oriented Fundamentals

Slides: 18

Outline summary: This foundational lecture introduces students to modern software engineering principles and the object-oriented programming paradigm. Students will explore the challenges of building industrial-scale software systems, understand why object technology has become the approach of choice for high-quality software, and learn fundamental OO concepts including classes, objects, encapsulation, and information hiding. The lecture also introduces Eiffel notation and provides an overview of the course structure.

- Understand the key challenges in modern software engineering and why systematic approaches are necessary
- Explain the role of object technology in building high-quality software systems
- Define and distinguish between classes and objects
- Apply encapsulation and information hiding principles to software design
- Read and interpret basic Eiffel notation
- Understand the course structure, assessment methods, and intended learning outcomes

Bloom alignment: 72 | Dominant: understand

- Apply-level gap: Objective requires applying encapsulation/information hiding to design, but slides only show completed examples. Missing: exercises where students decide what to make private/public in new scenarios, or refactor procedural code to use encapsulation.
- Apply-level gap: Objective requires reading/interpreting Eiffel, but slide 17 only labels syntax. Missing: unfamiliar Eiffel code snippets for students to trace, predict output, or identify what methods do.
- Analyze-level gap: For MSc students, 75% remember+understand distribution is high. Missing: comparative analysis of when to use encapsulation vs when overhead isn't justified, or diagnosing encapsulation violations in code.
- No evaluate or create activities: Students never assess design quality, critique encapsulation decisions, justify trade-offs, or design their own class structures during lecture.
- Passive code examples: Slides 15 and 17 present annotated code but don't require active interpretation. Students observe rather than apply understanding to decode behavior.